



NATIONAL UNIVERSITY
of Computer & Emerging Sciences

National University of Computer & Emerging Sciences

Computer Organization and Assembly Language Lab-1 Lab task 01

Name: Muhammad Zain

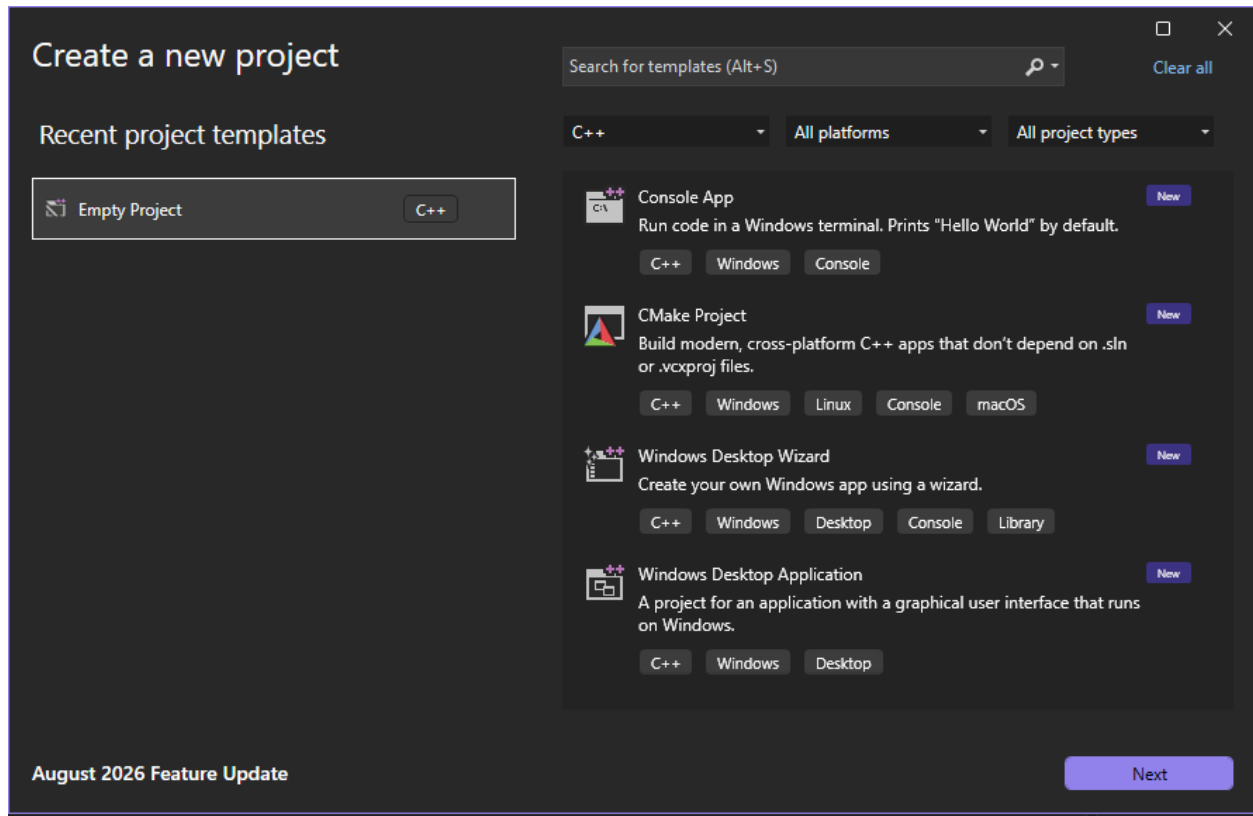
Section: BSE-3A

Roll number: 25K-3050

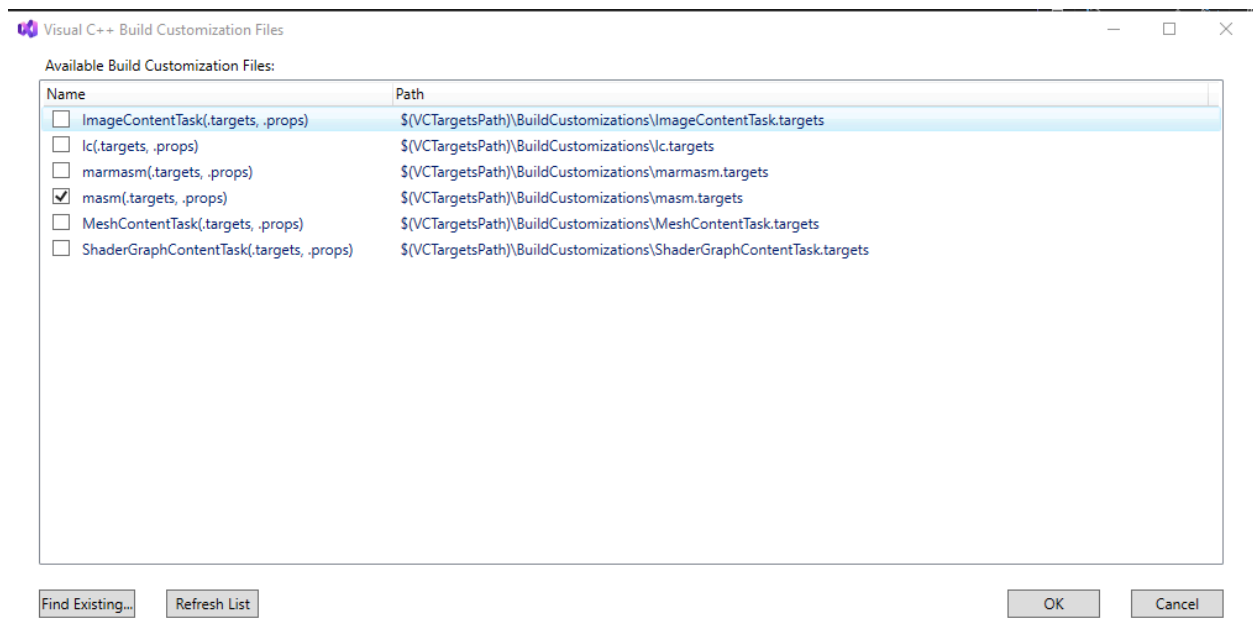
Instructor: Muhammad Nadeem

Lab Task 1:

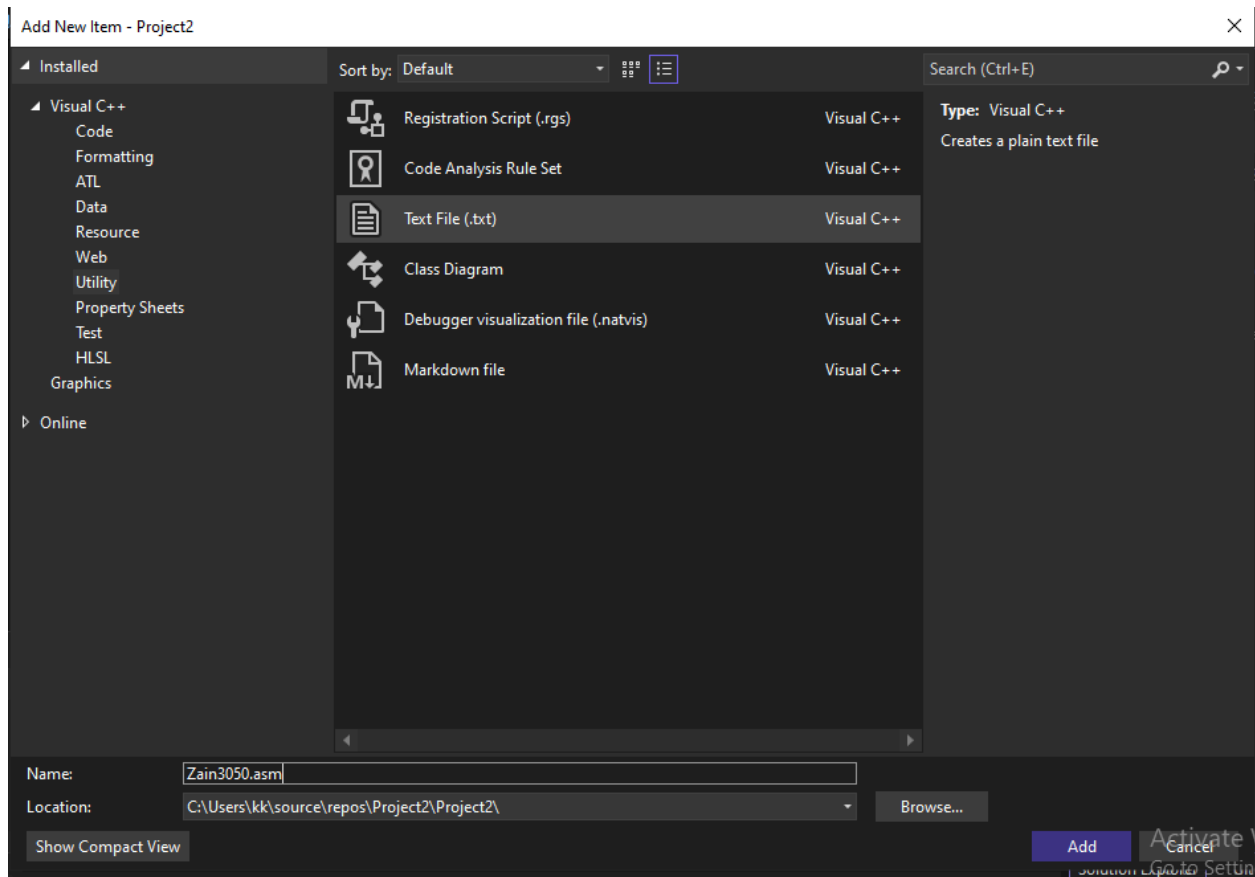
Selecting Project Type



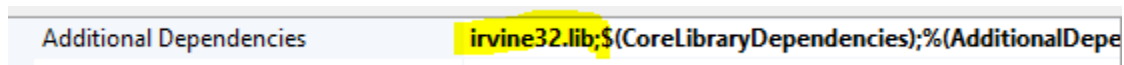
Selecting Masm at the Build customization option



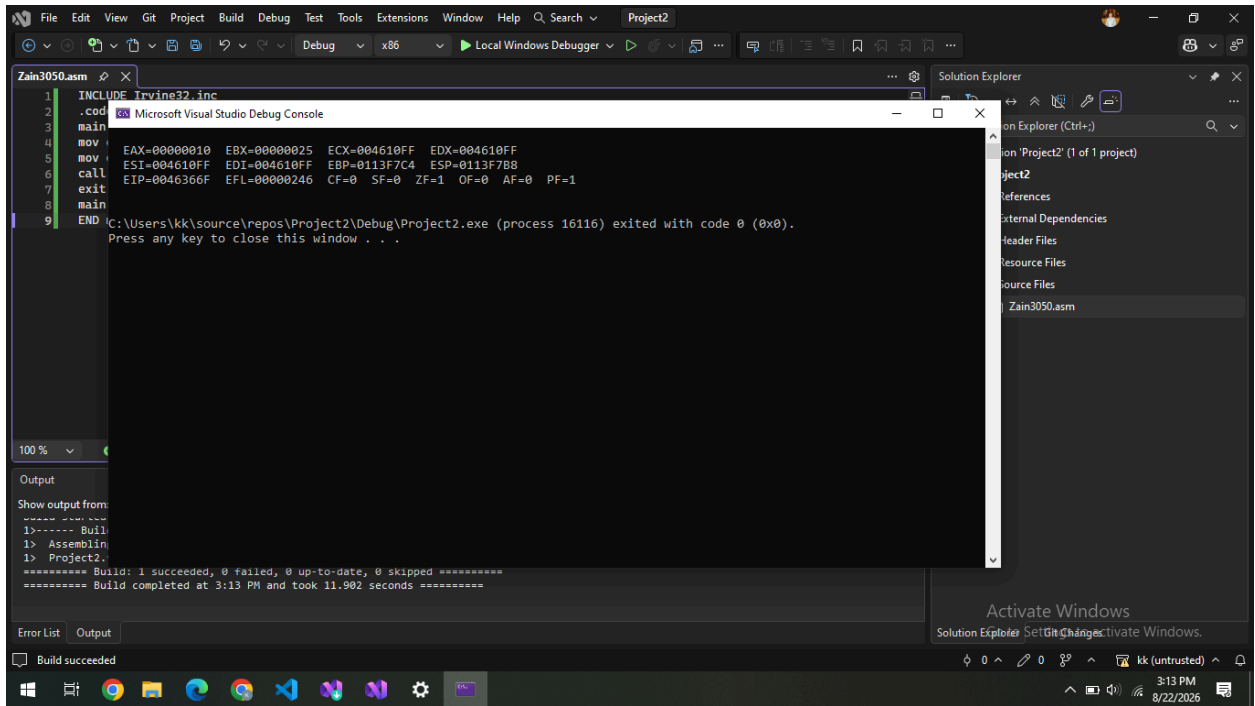
Adding a new file as asm.



Adding Directories Path at Highlighted Parts



Output



The screenshot shows the Visual Studio IDE with the assembly file `Zain3050.asm` open. The assembly code is as follows:

```
1 INCLUDE Irvine32.inc
2 .code
3 main
4 mov eax, 0
5 mov ebx, 0
6 call EIP=0046366F
7 exit
8 main
9 END
```

The Output window shows the following text:

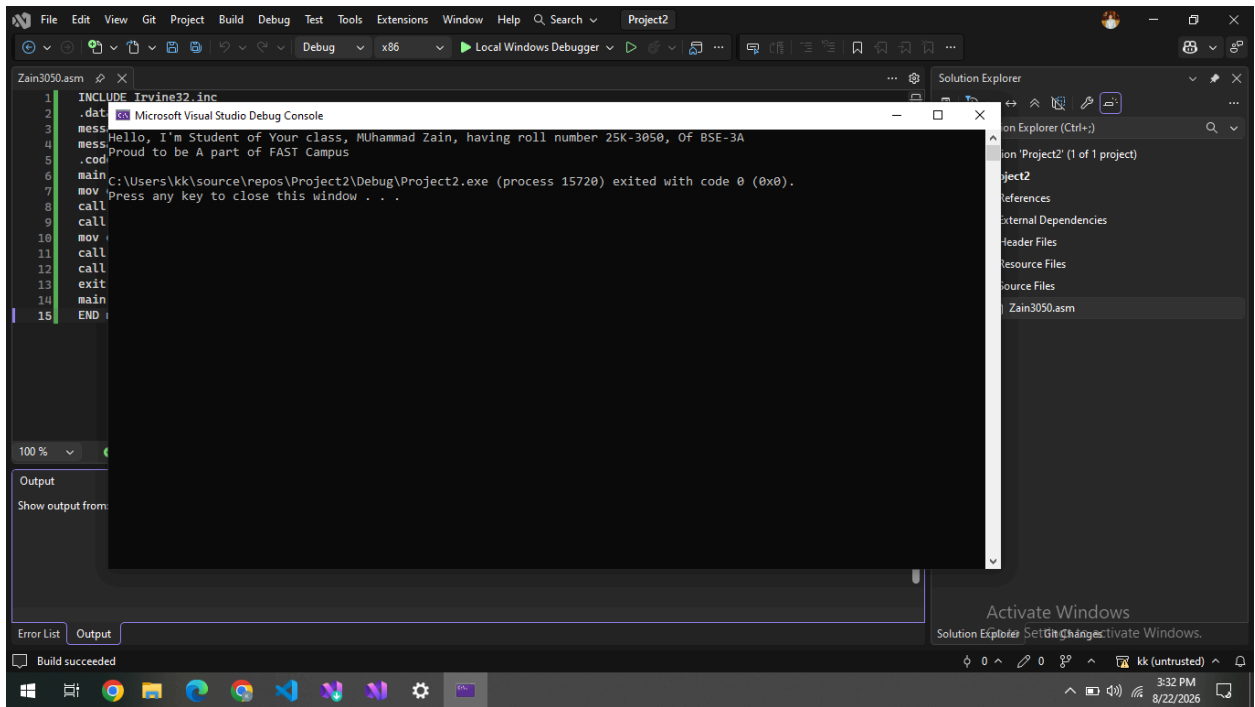
```
-----
1>----- Build
1> Assembling
1> Project2.
----- Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped -----
----- Build completed at 3:13 PM and took 11.902 seconds -----
```

The Microsoft Visual Studio Debug Console shows the following output:

```
C:\Users\kk\source\repos\Project2\Debug\Project2.exe (process 16116) exited with code 0 (0x0).
Press any key to close this window . . .
```

Lab Task 2:

Output



The screenshot shows the Visual Studio IDE with the assembly file `Zain3050.asm` open. The assembly code is as follows:

```
1 INCLUDE Irvine32.inc
2 .data
3 mess "Hello, I'm Student of Your class, MUHAMMAD Zain, having roll number 25K-3050, Of BSE-3A"
4 .code
5 mess "Proud to be A part of FAST Campus"
6 main
7 mov eax, 0
8 call EIP=0046366F
9 call EIP=0046366F
10 mov ebx, 0
11 call EIP=0046366F
12 call EIP=0046366F
13 exit
14 main
15 END
```

The Output window shows the following text:

```
-----
1>----- Build
1> Assembling
1> Project2.
----- Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped -----
----- Build completed at 3:13 PM and took 11.902 seconds -----
```

The Microsoft Visual Studio Debug Console shows the following output:

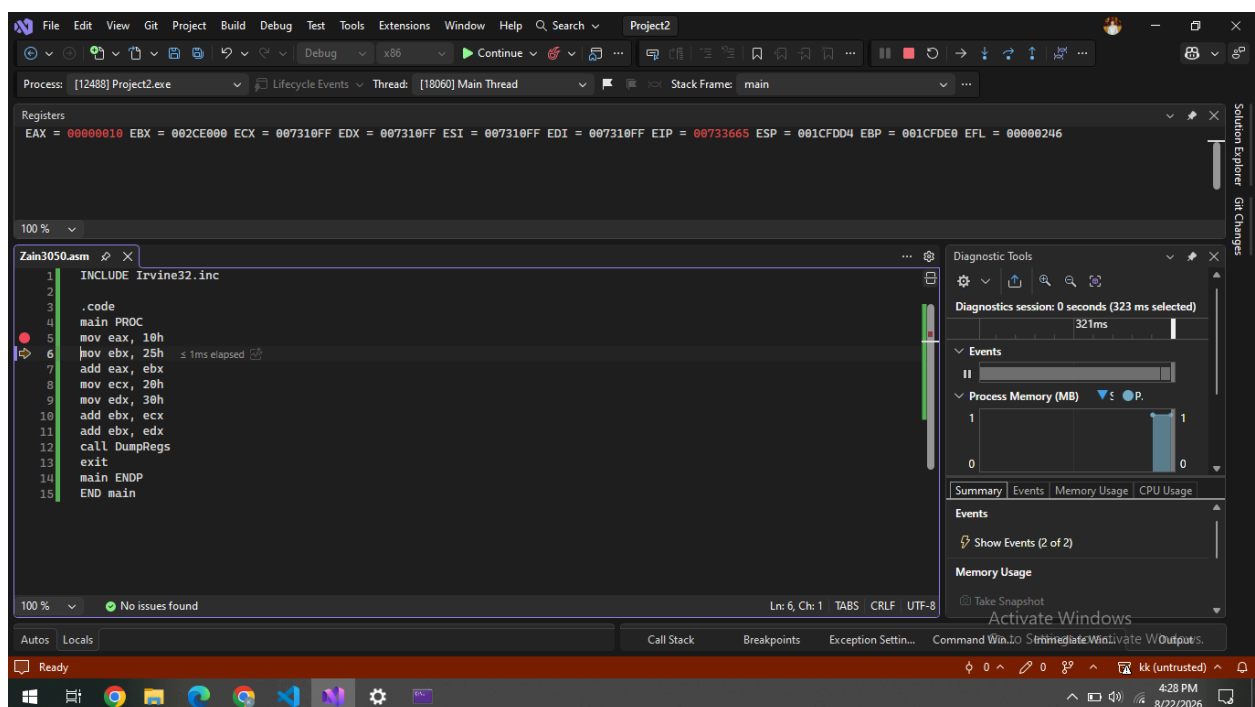
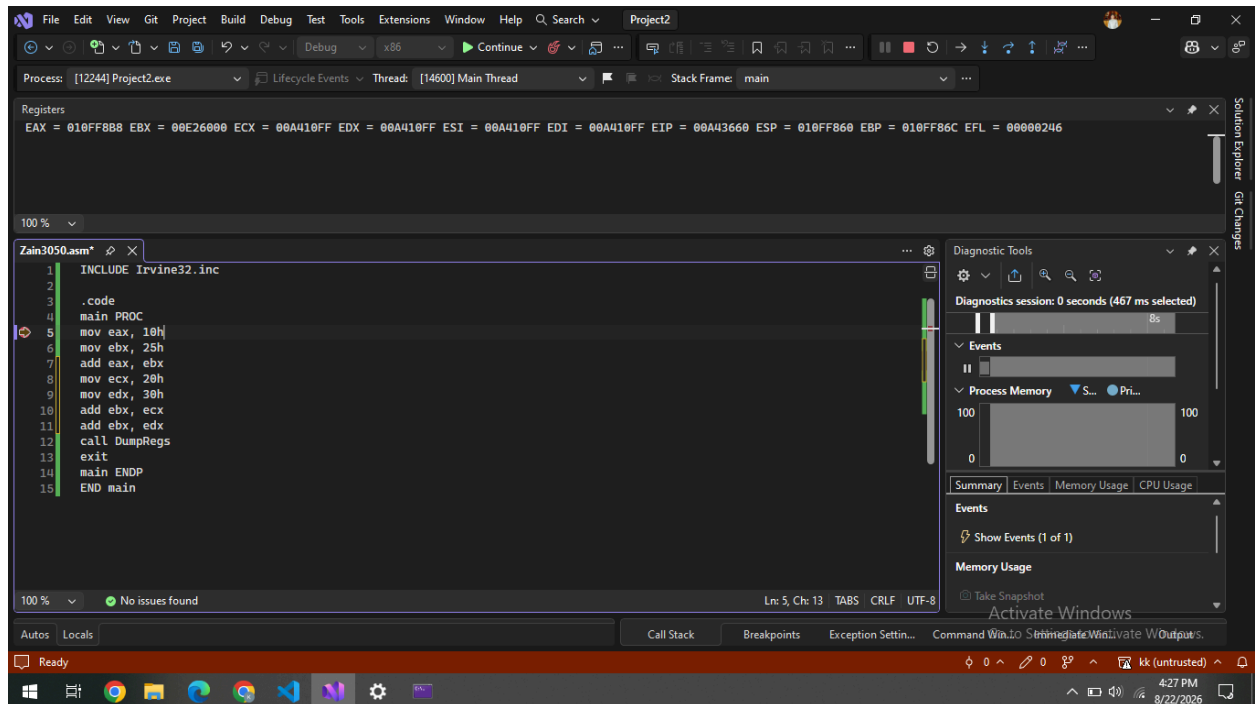
```
C:\Users\kk\source\repos\Project2\Debug\Project2.exe (process 15720) exited with code 0 (0x0).
Press any key to close this window . . .
```

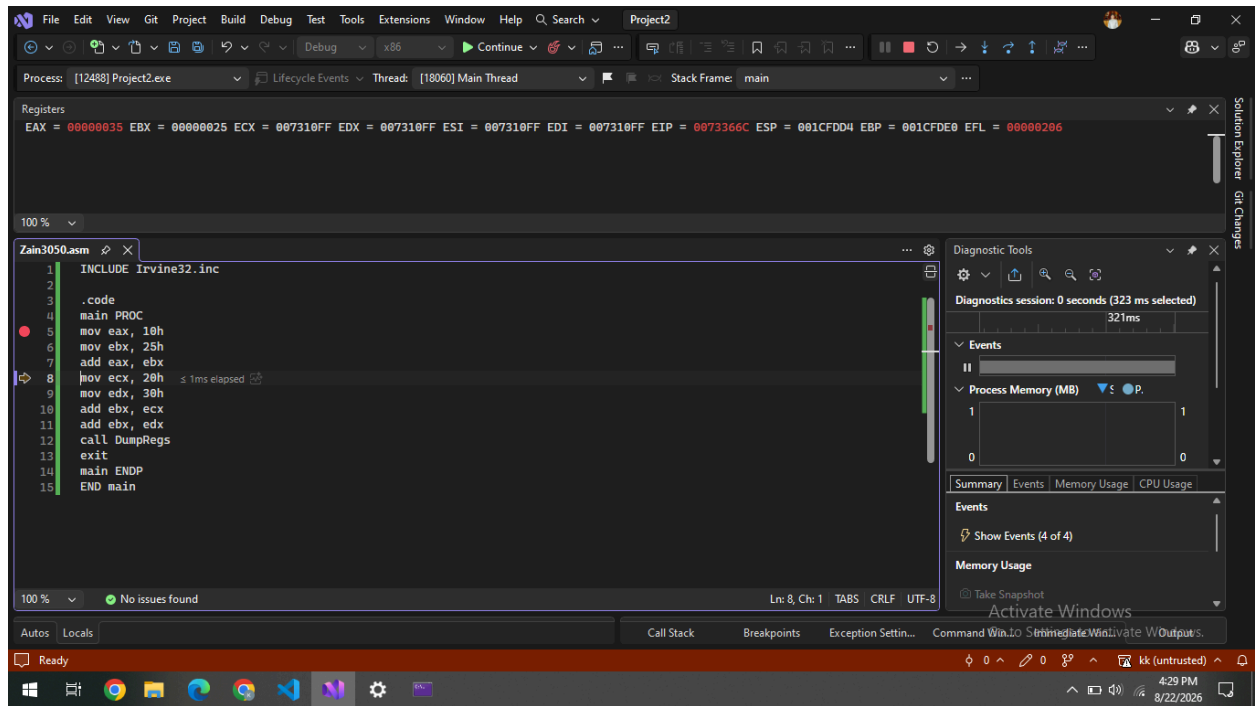
Code:

<https://github.com/muhammadzain-nu/Computer-Organization-and-Assembly-Language/tree/main/Lab%201%20Tasks>

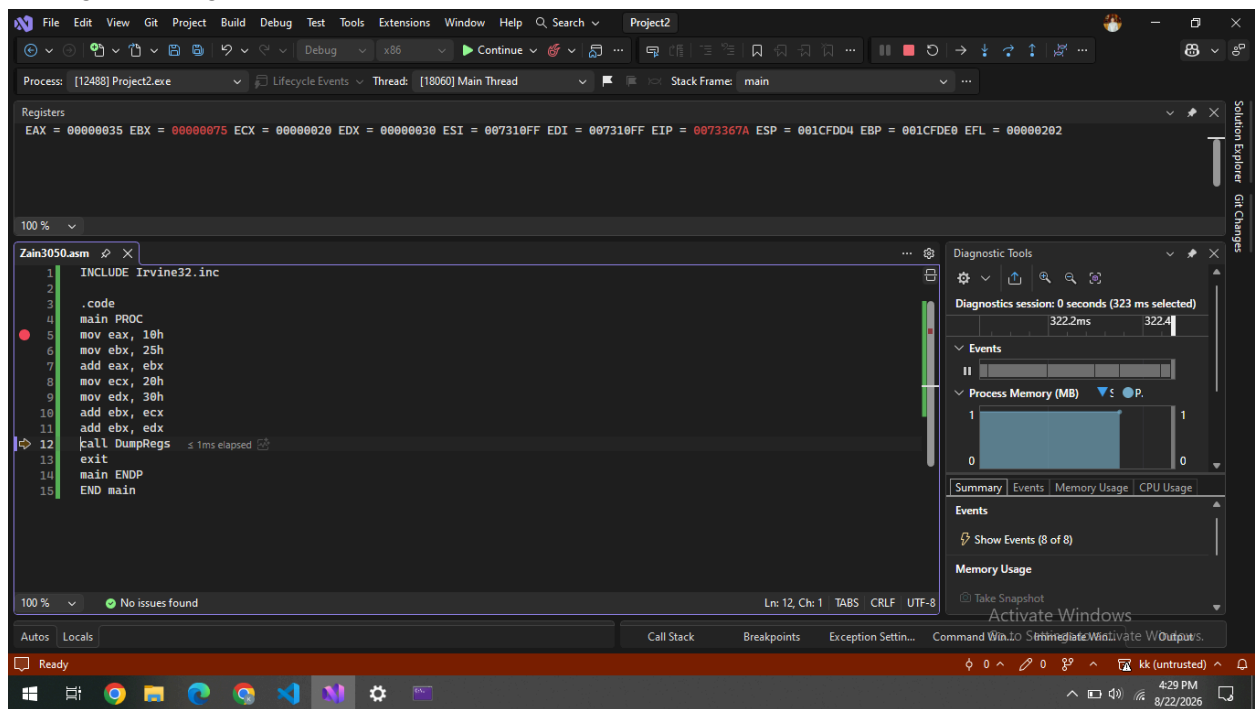
Lab Task 3:

Debugging:





Adding three registers/variables.



Table

State	EAX Value	EBX Value
Before ADD	10h	25h
After ADD	35h	25h

Role of DumpRegs: DumpRegs is a procedure from the Irvine32 library used specifically for debugging. When called, it prints the current hexadecimal values of all 32-bit general-purpose registers (EAX, EBX, ECX, EDX, ESI, EDI, EBP, and ESP) and the current state of the CPU's status flags (like the Zero, Sign, and Carry flags) to the console.

Purpose of MOV: The purpose of MOV is to copy the source file into the destination file. For example, `MOV eax, ebx`, the `eax` is the destination, which copies `ebx`, the source file.

Code:

<https://github.com/muhammadzain-nu/Computer-Organization-and-Assembly-Language/blob/main/Lab%201%20Tasks/Task%203.asm>

Lab Task 4:

Lab-01 was highly focused on configuring the environment, software, and the concept of how Irvine Library works and how its functions run in the environment. If we call `dumpregs`, that is a predefined function in `Irvine32.lib` that is used. I learnt about how a program includes the library Irvine, how to enter `.data` files, and in `.code` how we utilize functions and registers to add and mov. Microsoft Macro Assembler (MASM), which acts as the translator by converting human-readable assembly source code (`.asm`) into machine-readable object files (`.obj`). Once the code is assembled, the linker takes over to combine these generated object files with external resources—such as the `Irvine32.lib` library resolving any external dependencies and memory addresses to produce the final, runnable executable program (`.exe`).

Lab Task 5:

While we have C, C++, Java, and Python, like well-developed languages, which are useful for modern tools. Assembly language is essential for our understanding of computer architecture to learn how computers execute the tasks, manage memory, and utilize registers. This low-level hardware control is indispensable for performance optimization, allowing developers to write highly efficient, bare-metal code for resource-constrained embedded systems, operating system kernels, and performance-critical applications where every clock cycle matters. Ultimately, this knowledge reveals what happens when a compiler is used, leaving the programmer the ability to write better, more optimized, and easier-to-debug code in any higher-level language.